VEIL

Anonymous below threshold. Compliant above. Zero-knowledge proofs protect your spending.

Architecture Documentation

Privacy Payment Protocol on Sui

Version 2.0 (post-audit)

Sui Overflow 2026 — DeFi & Payments

BN254 Groth16 / Circom 2.1 / Sui Move 2024

Generated: May 2026

Table of Contents

---

1. Executive Summary

Veil is a privacy payment protocol on Sui that introduces **cumulative spending proofs** — a novel ZK primitive enabling users to transact anonymously below a regulatory threshold (CHF 1,000 per epoch, aligned with FINMA) while proving KYC compliance above it, without revealing amounts or identity. Each user's on-chain footprint is a single Poseidon commitment: a hash of their cumulative spending for the current epoch, updated atomically via Groth16 zero-knowledge proofs verified natively on Sui.

The protocol uses a UTXO-style commitment model with note-based nullifiers, enabling multiple transfers per epoch without linkability. The circuit enforces 11 constraints including domain-separated Poseidon hashes (3 tags), 64-bit range proofs on all arithmetic values, and threshold enforcement — all proven client-side in ~2 seconds via snarkjs WASM. The Move contract (`pool.move`) verifies proofs using Sui's native `sui::groth16` BN254 verifier, manages nullifier/commitment dynamic fields, and provides emergency controls via `AdminCap`. The architecture is designed for production deployment with a 1-epoch VK timelock, freeze mechanism, and privacy-preserving events that leak no sender, recipient, or amount information.

2. C4 Level 1 — System Context

The System Context diagram shows Veil's position in the broader ecosystem and identifies three actor types: Users who hold wallets and generate ZK proofs, Auditors/Regulators who can request selective disclosure (planned for Tier 2 via ElGamal), and Admins who hold the `AdminCap` object for pool governance.

docs/c4-context.html

Open in browser to view the interactive C4 Level 1 diagram

Actors

ACTOR	ROLE	INTERACTION
User	Wallet holder, proof generator	Deposits tokens, generates Groth16 proofs client-side, submits shielded transfers
Auditor / Regulator	Compliance oversight	Requests selective disclosure via ElGamal decryption (Tier 2, future)
Admin	Pool governance	Holds AdminCap: freeze/unfreeze pool, update VK (1-epoch timelock), emergency withdraw

External Systems

SYSTEM	TECHNOLOGY	ROLE
Sui Blockchain	L1, Move VM, BN254 verifier	Hosts contracts, verifies proofs natively, provides Clock for epochs
Browser	Next.js 14, snarkjs WASM	Runs frontend, generates proofs client-side (~2s), manages encrypted private state

3. C4 Level 2 — Container Diagram

The Container diagram decomposes Veil into its five runtime components, showing how data flows from circuit compilation through proof generation to on-chain verification.

docs/c4-container.html

Open in browser to view the interactive C4 Level 2 diagram

Containers

CONTAINER	TECHNOLOGY	RESPONSIBILITY
Sui Move Contracts	Move 2024 (pool.move, verifier.move, token.move)	Protocol core: shielded transfers, UTXO commitments, nullifier tracking, proof verification, token minting
Circom Circuit	Circom 2.1, BN254 Groth16	Defines 11 constraints (~1,250 R1CS), compiled to WASM + zkey for client-side proving
Frontend	Next.js 14, @mysten/dapp-kit	Wallet connection, proof generation (snarkjs WASM in Web Worker), encrypted localStorage state
Proof Converter	TypeScript (proof-converter.ts)	Converts snarkjs JSON proof to Sui arkworks compressed bytes (G2 compression, sign bit)
E2E Pipeline	TypeScript / Bun	End-to-end testing: deploy contracts, mint tokens, deposit, prove, verify on testnet

Data Flow

1. Circuit compiled to `.wasm` + `.zkey` (build artifacts) and `vk.json` (verifying key)

2. VK embedded in Pool at creation time (`transfer_vk` field)
3. Frontend loads WASM + zkey for client-side proving via snarkjs
4. User generates proof in browser (~2s), proof converter encodes to Sui bytes
5. Signed transaction calls `shielded_transfer(pool, proof_bytes, public_inputs_bytes, clock)`
6. On-chain: `verifier.move` calls `sui::groth16::verify_groth16_proof()` with BN254 curve

4. C4 Level 3 — Component Diagram (pool.move)

The Component diagram decomposes `pool.move` (254 lines) into its internal objects, entry functions, dynamic fields, internal helpers, and events. This is the core module that implements the shielded transfer protocol.

docs/c4-component.html

Open in browser to view the interactive C4 Level 3 diagram

Shared Objects

OBJECT	TYPE	FIELDS
Pool	Shared (has key)	`id: UID`, `balance: Balance<TOKEN>`, `transfer_vk: vector<u8>`, `threshold: u64`, `frozen: bool`, `pending_vk: vector<u8>`, `vk_update_epoch: u64`
AdminCap	Owned (has key)	`id: UID`, `pool_id: ID` (bound to specific pool)

Dynamic Fields

KEY TYPE	VALUE	PURPOSE
`NullifierKey { bytes }`	`bool`	Tracks spent nullifiers. Poseidon(2, secret, epoch, randOld). Permanent storage.
`CommitmentKey { bytes }`	`bool`	UTXO commitments. Consumed on spend, created on receive. Poseidon(1, cum, rand, secret).

Proof Verification Pipeline (shielded_transfer)

The 10-step pipeline in `shielded_transfer()`:

1. **Freeze check** — `assert!(!pool.frozen)`
2. **Input length** — `assert!(inputs.length() == 192)` (6 x 32 bytes)
3. **VK update** — Apply pending VK if current epoch >= `vk_update_epoch`
4. **Threshold check** — Extract LE u64 from bytes [64..72], assert == `pool.threshold`
5. **Epoch check** — Extract LE u64 from bytes [96..104], assert == `current_epoch(clock)`
6. **Groth16 verify** — `verifier::verify_transfer_proof(vk, proof, inputs)`
7. **Consume old commitment** — Assert exists, then `dynamic_field::remove()`
8. **Check nullifier** — Assert not already spent
9. **Store nullifier** — `dynamic_field::add(nullifier_key, true)`
10. **Create new commitment** — Assert not exists, then `dynamic_field::add()`

5. ZK Circuit Constraint Flow

The transfer circuit (`transfer.circom`) defines 11 constraints over 7 private and 6 public inputs. All hashes use Poseidon from circomlib with domain separation tags. The circuit compiles to ~1,250 R1CS constraints on the BN254 curve, verified on-chain via `sui::groth16`.

docs/c4-circuit.html

Open in browser to view the interactive circuit constraint flow diagram

Inputs

TYPE	SIGNAL	DESCRIPTION
Public [0]	`oldCommitment`	Poseidon(1, cumOld, randOld, userSecret)
Public [1]	`newCommitment`	Poseidon(1, cumNew, randNew, userSecret)
Public [2]	`threshold`	KYC-free epoch limit
Public [3]	`epochId`	Current epoch from on-chain Clock
Public [4]	`nullifier`	Poseidon(2, userSecret, epochId, randOld)
Public [5]	`txAmountHash`	Poseidon(3, txAmount, salt)
Private	`cumulativeOld`	Previous cumulative spending this epoch
Private	`cumulativeNew`	cumOld + txAmount
Private	`txAmount`	This transaction's amount
Private	`randomnessOld`	Blinding factor for old commitment
Private	`randomnessNew`	Blinding factor for new commitment

TYPE	SIGNAL	DESCRIPTION
Private	`userSecret`	Master secret (never revealed)
Private	`salt`	Salt for txAmountHash

Constraints

#	CATEGORY	CONSTRAINT	COMPONENT
C1	Commitment	`Poseidon(1, cumOld, randOld, secret) === oldCommitment`	Poseidon(4)
C2	Commitment	`Poseidon(1, cumNew, randNew, secret) === newCommitment`	Poseidon(4)
C3	Arithmetic	`cumNew === cumOld + txAmount`	Addition
C4	Arithmetic	`txAmount > 0`	GreaterThan(64)
C5	Range	`cumulativeOld in [0, 2^64)`	Num2Bits(64)
C6	Range	`txAmount in [0, 2^64)`	Num2Bits(64)
C7	Range	`cumulativeNew in [0, 2^64)`	Num2Bits(64)
C8	Range	`threshold in [0, 2^64)`	Num2Bits(64)
C9	Threshold	`cumNew <= threshold`	LessEqThan(64)
C10	Nullifier	`Poseidon(2, secret, epoch, randOld) === nullifier`	Poseidon(4)
C11	Amount Hash	`Poseidon(3, txAmount, salt) === txAmountHash`	Poseidon(3)

Domain Separation Tags

TAG	DOMAIN	HASH FUNCTION
1	Commitment hashes	`Poseidon(1, cumulative, randomness, userSecret)`
2	Nullifier hashes	`Poseidon(2, userSecret, epochId, randomnessOld)`
3	Amount hashes	`Poseidon(3, txAmount, salt)`

Audit Fixes (v2)

- **CRYPTO-004:** Commitments now bound to `userSecret` via Poseidon(4) — prevents commitment theft
- **CRYPTO-006:** Note-based nullifiers include `randomnessOld` — enables multiple transfers per epoch
- **CRYPTO-011:** txAmountHash uses domain tag 3 — prevents cross-domain hash collision

6. Security Properties

PROPERTY	MECHANISM	ERROR CODE
Amount privacy	Amounts never appear in public inputs. Only Poseidon commitment hashes are stored on-chain.	—
Sender privacy	No address in transfer events. Commitments are pseudonymous. Events emit only nullifier + new commitment.	—
Replay prevention	Nullifier stored in dynamic field after use. Deterministic: Poseidon(2, secret, epoch, randOld).	`E_NULLIFIER_SPENT (2)`
Overflow prevention	Num2Bits(64) applied independently to `cumulativeOld`, `txAmount`, `cumulativeNew`, and `threshold`.	—
Epoch binding	Nullifier includes `epochId` from on-chain Clock. Contract verifies proof epoch matches chain epoch.	`E_EPOCH_MISMATCH (8)`
Domain separation	Three Poseidon tags (1=commit, 2=nullifier, 3=amount). No cross-type hash collision possible.	—
Commitment chain	UTXO model: old commitment must exist (consumed), new	`E_COMMITMENT_CHAIN_BROKEN (9)`, `E_COMMITMENT_EXISTS (10)`

PROPERTY	MECHANISM	ERROR CODE
	commitment must not exist (created).	
VK integrity	`sui::groth16` internally enforces `gamma_g2 != delta_g2`, preventing FOOM-style VK attacks.	`E_INVALID_PROOF (3)`
VK update safety	1-epoch timelock via `propose_vk_update()`. Applied only on next `shielded_transfer` after epoch boundary.	`E_VK_UPDATE_LOCKED (12)`
Emergency stop	`freeze_pool()` / `unfreeze_pool()` via AdminCap. All entry functions check `pool.frozen` first.	`E_FROZEN (1)`
Dust prevention	Minimum deposit of 0.001 TOKEN (`MIN_DEPOSIT = 1_000` at 6 decimals).	`E_DUST_DEPOSIT (11)`
Trusted setup	Hermez Powers of Tau (pot15). Dev contribution for testing. Production requires ceremony.	—

7. API Reference — Contract Functions

veil::pool

FUNCTION	ACCESS	PARAMETERS	DESCRIPTION
`create_pool`	public	`transfer_vk, threshold, ctx`	Creates shared Pool + AdminCap. AdminCap sent to caller.
`deposit_and_register`	public	`pool, coin, commitment, ctx`	Deposit tokens + register genesis commitment. Anti-griefing: requires MIN_DEPOSIT.
`deposit`	public	`pool, coin, ctx`	Deposit additional tokens (no commitment registration).
`shielded_transfer`	public	`pool, proof_bytes, public_inputs_bytes, clock, ctx`	Core privacy transfer: verify proof, consume old commitment, store nullifier, create new commitment.

FUNCTION	ACCESS	PARAMETERS	DESCRIPTION
`withdraw`	public (AdminCap)	`pool, cap, amount, recipient, ctx`	Emergency withdraw. AdminCap-gated. Transfers tokens to recipient.
`propose_vk_update`	public (AdminCap)	`pool, cap, new_vk, clock`	Stage VK update with 1-epoch timelock. Applied on next shielded_transfer.
`freeze_pool`	public (AdminCap)	`pool, cap`	Emergency stop. Sets `pool.frozen = true`.
`unfreeze_pool`	public (AdminCap)	`pool, cap`	Resume operations. Sets `pool.frozen = false`.

veil::pool — View Functions

FUNCTION	RETURNS	DESCRIPTION
`is_frozen(pool)`	`bool`	Current freeze state
`pool_balance(pool)`	`u64`	Total shielded balance
`threshold(pool)`	`u64`	KYC-free epoch limit
`current_epoch(clock)`	`u64`	timestamp_ms / 2,592,000,000

veil::verifier

FUNCTION	ACCESS	PARAMETERS	DESCRIPTION
`verify_transfer_proof`	public(package)	`vk_bytes`, `proof_bytes`, `public_inputs_bytes`	Wraps `sui::groth16` BN254 verification. Returns `bool`.

veil::token

FUNCTION	PARAMETERS	DESCRIPTION
`mint`	`treasury, amount, recipient, ctx`	Mint arbitrary amount to recipient
`faucet`	`treasury, ctx`	Mint 1,000 VEIL (1_000_000_000 base units) to caller

Error Codes

CODE	NAME	TRIGGER
1	`E_FROZEN`	Pool is frozen
2	`E_NULLIFIER_SPENT`	Nullifier already used (replay attempt)
3	`E_INVALID_PROOF`	Groth16 verification failed
4	`E_NOT_POOL_ADMIN`	AdminCap.pool_id does not match pool
5	`E_THRESHOLD_MISMATCH`	Proof threshold != pool threshold
6	`E_INSUFFICIENT_BALANCE`	Withdraw amount exceeds pool balance
7	`E_INVALID_INPUTS_LENGTH`	Public inputs != 192 bytes or commitment != 32 bytes
8	`E_EPOCH_MISMATCH`	Proof epoch != on-chain Clock epoch
9	`E_COMMITMENT_CHAIN_BROKEN`	Old commitment not found in pool

CODE	NAME	TRIGGER
10	`E_COMMITMENT_EXISTS`	New commitment already exists (collision)
11	`E_DUST_DEPOSIT`	Deposit below MIN_DEPOSIT (0.001 TOKEN)
12	`E_VK_UPDATE_LOCKED`	VK update already pending
13	`E_NULLIFIER_NOT_CANONICAL`	Nullifier bytes not canonical

8. Public Input Byte Layout

The Sui verifier reads public inputs as a flat byte array. Each input is 32 bytes (little-endian field element on BN254 scalar field). Total: 192 bytes (6 inputs x 32 bytes).

OFFSET	FIELD	INDEX	ON-CHAIN USAGE
`0..32`	oldCommitment	0	Consumed from CommitmentKey dynamic field
`32..64`	newCommitment	1	Created as new CommitmentKey dynamic field
`64..96`	threshold	2	LE u64 at [64..72], upper bytes asserted zero
`96..128`	epochId	3	LE u64 at [96..104], compared to Clock epoch
`128..160`	nullifier	4	Full 32 bytes stored as NullifierKey
`160..192`	txAmountHash	5	Not extracted on-chain (receiver-side)

9. Epoch Design

Spending counters reset every 30 days at epoch boundaries. The epoch ID is deterministically derived from the on-chain `Clock` object's timestamp.

```
epoch_id = Clock::timestamp_ms() / EPOCH_DURATION_MS
EPOCH_DURATION_MS = 2,592,000,000  (30 days in milliseconds)
```

Epoch Boundary Behavior

PROPERTY	VALUE
Duration	30 days (2,592,000,000 ms)
Genesis commitment	`Poseidon(1, 0, 0, userSecret)` — deterministic per user, no interaction required
Genesis nullifier	N/A — genesis commitment registered via `deposit_and_register()`
Cumulative reset	`cumulativeOld = 0` at new epoch start
Cross-epoch linkability	None — new commitment with fresh randomness, no linkage to previous epoch

Tiered Compliance

TIER	CONDITION	PRIVACY LEVEL	COMPLIANCE
0	Cumulative < CHF 1,000/epoch	Fully anonymous	None required
1	Cumulative >= CHF 1,000/epoch	Identity hidden	ZK proof of KYC credential (compliance circuit, ~7,200 constraints)
2	Regulatory request	Auditor view only	ElGamal selective disclosure (future stretch goal)

Veil — Privacy Payment Protocol on Sui

Sui Overflow 2026 — DeFi & Payments Track

MIT License